Киселев ГП, ИВТ-1.1, s3-storage

ОТЧЁТ

Тема: Работа с Yandex Object Storage (S3-совместимое хранилище)

Студент: Киселев Георгий
Группа: ИВТ-1.1

Цель работы

Научиться создавать и настраивать объектное хранилище в Yandex Cloud, реализовать на языке Python программу для выполнения основных операций с объектами в хранилище:

- получение списка загруженных файлов;
- загрузка файла в бакет;
- скачивание файла из бакета;
- удаление файла из бакета.

---

1. Ход выполнения работы

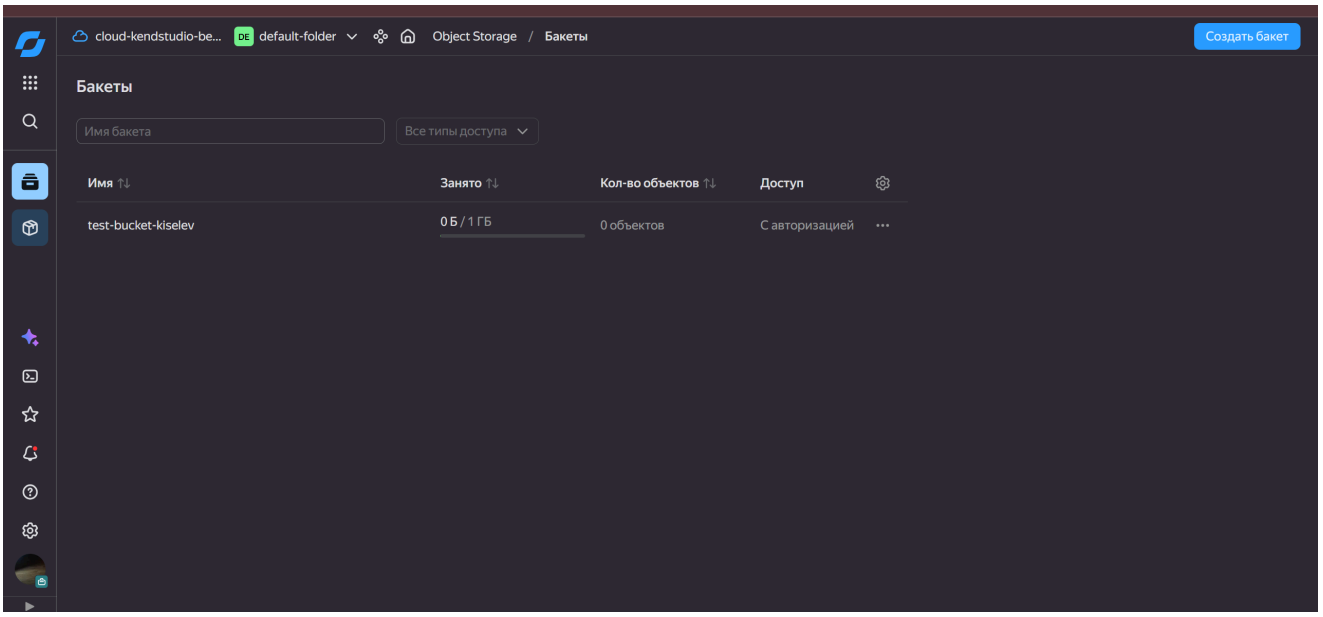
1.1 Создание бакета в Yandex Object Storage

Действие: В консоли Yandex Cloud был создан бакет (хранилище) для работы с файлами.

Параметры бакета:

- **Имя бакета:** `test-bucket-kiselev`
- **Класс хранилища:** Стандартное (для использования бесплатного тарифа)
- **Тип доступа:** Ограниченный (с авторизацией)

Результат на скриншоте №1:



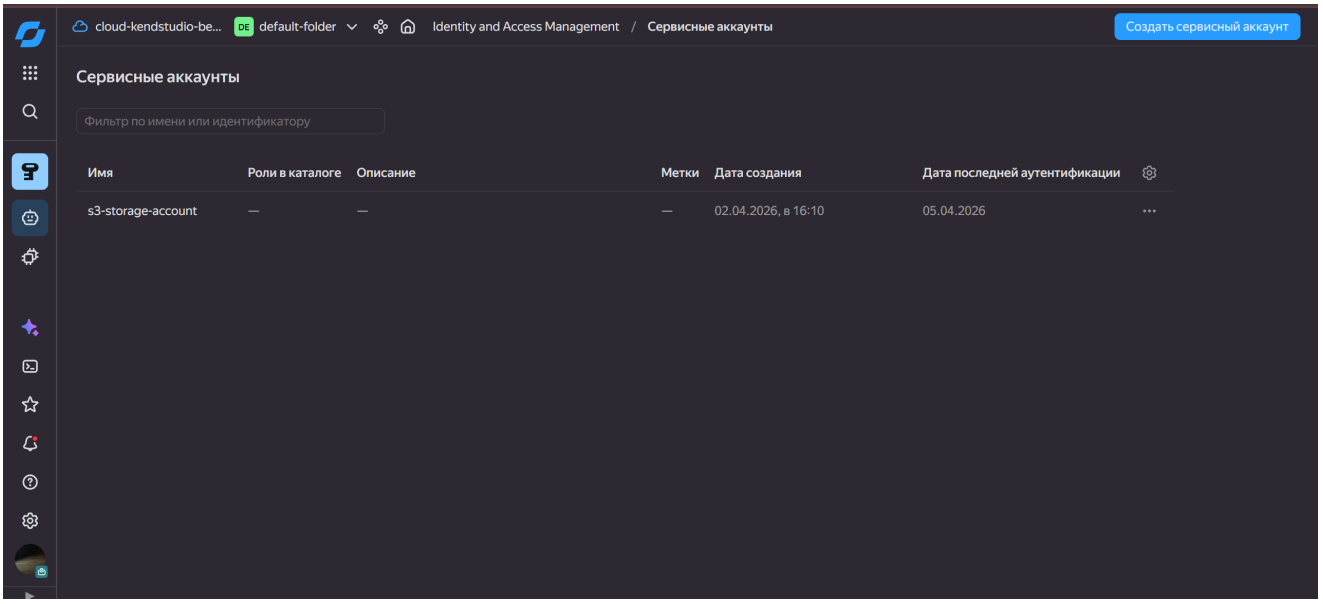
1.2 Создание сервисного аккаунта

Действие: Был создан сервисный аккаунт, который будет использоваться программой для авторизации в Yandex Cloud.

Параметры сервисного аккаунта:

- **Имя:** `s3-storage-account`
- **Дата создания:** 02.04.2026

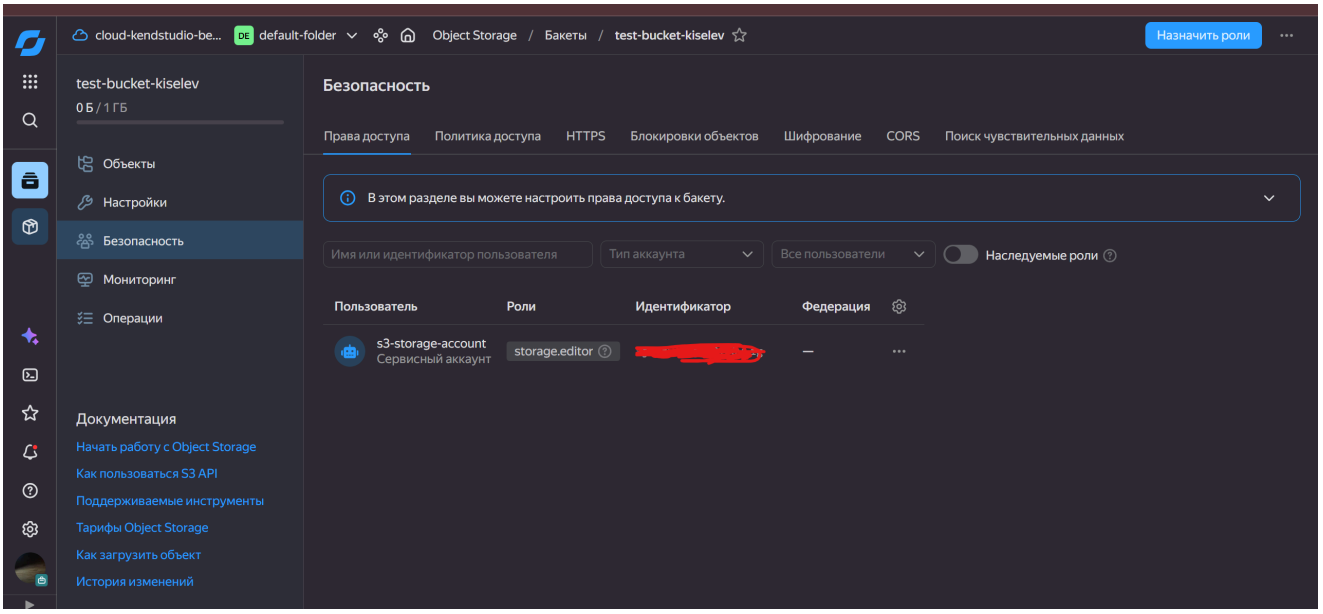
Результат на скриншоте №2:



1.3 Назначение роли storage.editor

Действие: Сервисному аккаунту была назначена роль `storage.editor`, которая даёт права на чтение и запись в бакете.

Результат на скриншоте №3:



1.4 Создание статического ключа доступа

Действие: Для сервисного аккаунта был создан статический ключ доступа, необходимый для аутентификации приложения.

Полученные данные (сохранены в файле `.env`):

```
AWS_ACCESS_KEY_ID = YCAJEW_2VAs_pLXA1UhD1gXiD
AWS_SECRET_ACCESS_KEY = ***скрыто в целях безопасности***
ENDPOINT_URL = https://storage.yandexcloud.net
BUCKET_NAME = test-bucket-kiselev
```

1.5 Настройка локального окружения

Установленные библиотеки Python:

```
pip install boto3 python-dotenv
```

Структура проекта:

```
s3-project/
├── s3_manager.py      # основная программа
```

```
├── .env                # файл с ключами доступа (скрыт)
```

1.6 Реализация программы `s3_manager.py`

Программа реализует класс `S3StorageManager` со следующими методами:

Метод	Описание	Параметры
`__init__()`	Инициализация подключения к Yandex Object Storage	—
`list_files()`	Получение списка всех файлов в бакете	—
`upload_file()`	Загрузка файла в бакет (класс STANDARD)	`local_file_path`, `object_name`
`download_file()`	Скачивание файла из бакета	`object_name`, `local_file_path`
`delete_file()`	Удаление файла из бакета	`object_name`

Листинг программы `s3_manager.py`:

```python
import os
import boto3
from botocore.exceptions import ClientError
from dotenv import load_dotenv

# Загружаем ключи из файла .env
load_dotenv()

class S3StorageManager:
    def __init__(self):
        """Подключение к Яндекс Облаку"""
        self.bucket_name = os.getenv('BUCKET_NAME')
        session = boto3.session.Session()
        self.s3_client = session.client(
            service_name='s3',
            endpoint_url=os.getenv('ENDPOINT_URL'),
            aws_access_key_id=os.getenv('AWS_ACCESS_KEY_ID'),
            aws_secret_access_key=os.getenv('AWS_SECRET_ACCESS_KEY'),
            region_name='ru-central1'
        )

    def list_files(self):
        """Показать все файлы в бакете"""
        try:
```

```
            response =
self.s3_client.list_objects_v2(Bucket=self.bucket_name)
            if 'Contents' not in response:
                print("В бакете нет файлов.")
                return []

            file_list = [item['Key'] for item in response['Contents']]
            print(f"Найдено файлов: {len(file_list)}")
            for file_name in file_list:
                print(f"   - {file_name}")
            return file_list
        except ClientError as e:
            print(f"! - Ошибка: {e}")
            return []

    def upload_file(self, local_file_path, object_name=None):
        """Загрузить файл в облако"""
        if object_name is None:
            object_name = os.path.basename(local_file_path)

        try:
            self.s3_client.upload_file(
                local_file_path,
                self.bucket_name,
                object_name,
                ExtraArgs={'StorageClass': 'STANDARD'}
            )
            print(f"Файл '{local_file_path}' загружен как '{object_name}'")
            return True
        except FileNotFoundError:
            print(f"! - Файл '{local_file_path}' не найден!")
            return False
        except ClientError as e:
            print(f"! - Ошибка загрузки: {e}")
            return False

    def download_file(self, object_name, local_file_path=None):
        """Скачать файл из облака"""
        if local_file_path is None:
            local_file_path = object_name

        try:
            self.s3_client.download_file(self.bucket_name, object_name,
local_file_path)
            print(f"Файл '{object_name}' скачан как '{local_file_path}'")
            return True
        except ClientError as e:
            print(f"! - Ошибка скачивания: {e}")
            return False
```

```
    def delete_file(self, object_name):
        """Удалить файл из облака"""
        try:
            self.s3_client.delete_object(Bucket=self.bucket_name,
Key=object_name)
            print(f"Файл '{object_name}' удалён")
            return True
        except ClientError as e:
            print(f"! - Ошибка удаления: {e}")
            return False

def main():
    print("=" * 50)
    print("---РАБОТА С YANDEX OBJECT STORAGE---")
    print("=" * 50)

    storage = S3StorageManager()

    # 1. Создаём тестовый файл
    print("\n1. Создаём тестовый файл...")
    with open("test.txt", "w", encoding="utf-8") as f:
        f.write("Привет! Это тестовый файл.")
    print("Файл test.txt создан")

    # 2. Загружаем файл
    print("\n2. Загружаем файл в облако...")
    storage.upload_file("test.txt")

    # 3. Смотрим список файлов
    print("\n3. Список файлов в облаке:")
    storage.list_files()

    # 4. Скачиваем файл
    print("\n4. Скачиваем файл из облака...")
    storage.download_file("test.txt", "downloaded_test.txt")

    # 5. Удаляем файл
    print("\n5. Удаляем файл из облака...")
    storage.delete_file("test.txt")

    # 6. Проверяем, что удалилось
    print("\n6. Проверяем список после удаления:")
    storage.list_files()

    # Удаляем локальные файлы
    os.remove("test.txt")
    if os.path.exists("downloaded_test.txt"):
        os.remove("downloaded_test.txt")

    print("\n" + "=" * 50)
```

```python
    print("---ВСЕ ОПЕРАЦИИ ВЫПОЛНЕНЫ УСПЕШНО!---")
    print("=" * 50)

if __name__ == "__main__":
    main()
```

1.7 Запуск программы и демонстрация работы

Команда запуска:

```
python s3_manager.py
```

Результат выполнения на скриншоте №4:

```
==================================================
---РАБОТА С YANDEX OBJECT STORAGE---
==================================================

1. Создаём тестовый файл...
Файл test.txt создан

2. Загружаем файл в облако...
Файл 'test.txt' загружен как 'test.txt'

3. Список файлов в облаке:
Найдено файлов: 1
   - test.txt

4. Скачиваем файл из облака...
Файл 'test.txt' скачан как 'downloaded_test.txt'

5. Удаляем файл из облака...
Файл 'test.txt' удалён

6. Проверяем список после удаления:
В бакете нет файлов.

==================================================
---ВСЕ ОПЕРАЦИИ ВЫПОЛНЕНЫ УСПЕШНО!---
==================================================
PS C:\Users\Георгий\Desktop\cmp_practicum>
```

---

2. Результаты выполнения операций

№	Операция	Результат	Подтверждение
1	Создание локального тестового файла	Успешно	`Файл test.txt создан`
2	Загрузка файла в бакет	Успешно	`Файл 'test.txt' загружен как 'test.txt'`
3	Получение списка файлов	Успешно	`Найдено файлов: 1` → `test.txt`

№	Операция	Результат	Подтверждение
4	Скачивание файла из бакета	Успешно	`Файл 'test.txt' скачан как 'downloaded_test.txt'`
5	Удаление файла из бакета	Успешно	`Файл 'test.txt' удалён`
6	Проверка списка после удаления	Успешно	`В бакете нет файлов`

---

3. Выводы

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. **Создан бакет** `test-bucket-kiselev` в Yandex Object Storage с классом хранения "Стандартное", что позволяет использовать бесплатный тариф (1 ГБ дискового пространства и до 10 000 операций).
2. **Создан сервисный аккаунт** `s3-storage-account`, который используется для безопасной авторизации приложения.
3. **Назначена роль** `storage.editor`, предоставляющая необходимые права на операции с бакетом.
4. **Получен статический ключ доступа** (Access Key ID и Secret Key) для аутентификации.
5. **Разработана программа** на языке Python с использованием библиотеки `boto3`, которая реализует все требуемые функции:
   - `upload_file()` — загрузка файла в бакет;
   - `list_files()` — получение списка файлов;
   - `download_file()` — скачивание файла из бакета;
   - `delete_file()` — удаление файла из бакета.
6. **Программа успешно протестирована** — все операции выполнены корректно, ошибок не возникло.
7. **Важно:** В коде программы явно указан параметр `StorageClass='STANDARD'`, что гарантирует использование стандартного хранилища и попадание в бесплатный уровень тарификации Yandex Cloud.